

LeetCode Dashboard Review Report

Project review summary for `leetcode-dashboard` • Generated on 2026-04-08

This report summarizes the main issues found in the current backend implementation, with a focus on efficiency, reliability, and error handling.

Key Issues

1. POTD null-safety risk

`dailyProblemHolder.getCurrentProblem()` can be null if the scheduler has not run yet or if the fetch failed. That makes `/ispotdsolved/{username}` vulnerable to a null pointer.

2. N+1 remote calls in recent solved problems

`getRecentSolvedProblems()` fetches submission history and then makes one additional LeetCode GraphQL request per accepted submission to fetch question details. This grows poorly for active users and increases latency and rate-limit risk.

3. User stats are cached forever

Once a student record exists, `getUserStats()` returns the DB value without refreshing from LeetCode. Any stale or incorrect ranking, streak, or solved counts can remain stuck in the database indefinitely.

4. GraphQL error handling is incomplete

The code parses `data` directly but does not consistently inspect GraphQL `errors`, null payloads, or partial responses. This can silently convert upstream failures into misleading defaults such as `0`, `0.00`, or empty values.

5. Exception messages are too broad

Multiple unrelated failures are wrapped into generic runtime exceptions like `"User not exists"` or similar messages. The global handler also maps every runtime exception to HTTP 400, which hides whether a failure came from invalid input, upstream API instability, or a server bug.

6. Username encoding is handled incorrectly

The code replaces spaces with `%20` before storage and lookup. This can create inconsistent DB keys and break future queries. Usernames should be stored in canonical raw form, while transport encoding should be handled by the HTTP client layer.

7. Blocking HTTP without timeout or retry

WebClient is used in a blocking style throughout the service, but there are no request timeouts, retries, or structured fallback behavior. If LeetCode slows down or returns anti-bot responses, request threads can block and fail unpredictably.

Recommended Improvements

- Add a shared GraphQL response validator that checks `errors`, missing `data`, and null nodes before parsing.
- Refresh stored stats using a TTL or a force-refresh path instead of returning cached rows forever.
- Protect POTD reads by fetching on demand when the in-memory holder is empty.
- Deduplicate accepted submission slugs and avoid repeated question-detail requests for problems already stored locally.
- Introduce custom exceptions such as `UserNotFoundException`, `ExternalApiException`, and `DataParsingException`.
- Return more accurate HTTP statuses like 404 for missing users and 502 for upstream LeetCode failures.
- Add timeout and retry policies to WebClient and replace `System.out.println` with structured logging.
- Stop manual username percent-encoding before persistence.

Priority Order

1. Fix GraphQL null/error handling.
2. Add POTD null-safety.
3. Improve cache refresh behavior for user stats.
4. Reduce N+1 LeetCode requests in recent solved problems.
5. Upgrade exception modeling and HTTP status mapping.